

Compiled by ab

Computer Graphics (CSIT 401)

III semester, BScCSIT

Compiled by **Ankit Bhattarai**
Ambition Guru College

Syllabus

Unit	Contents	Hours	Remarks
1.	Introduction to Computer Graphics	5	
2.	Scan Conversion Algorithms	7	<i>Very Important</i>
3.	2D Transformations and Viewing	6	<i>Very Important</i>
4.	3D Transformations and Viewing	5	
5.	3D Object Representation	6	
6.	Solid Modeling	3	
7.	Visible Surface Detection	5	
8.	Illumination, Shading and Rendering Techniques	5	
9.	OpenGL and Graphics Programming	3	

Practical Works

Credit hours : 3

Compiled by ab

Unit 2

Scan Conversion

Algorithm

(7 hrs.)

Introduction, Basic graphics primitives, Line drawing algorithms: DDA, Bresenham's Line Drawing Algorithm with derivations, numerical example and implementation, Circle and ellipse drawing: Midpoint algorithms with derivations and numerical examples.

Polygons, Polygon fill areas and fill-area primitives, polygon representation and filling algorithms, Inside-outside test, Boundary-fill and Flood-fill algorithm

Scan Conversion

- Scan conversion, also known as rasterization, is the fundamental process of converting continuous, mathematical vector graphics (such as points, lines, circles, and polygons) into a discrete grid of pixels that can be displayed on a screen or printed.

Output Primitive and Line Scan Conversion

Output primitives

- They are the basic primitive's structure such as points, straight line segments, circles, curves and surfaces which are used to describe a scene.

Line Scan Conversion

The slope-intercept equation of a straight line is: $y = mx + b$

where m = slope of line and, b = y-intercept.

For any two given points (x_1, y_1) and (x_2, y_2)

$$\text{slope } (m) = \frac{y_2 - y_1}{x_2 - x_1}$$

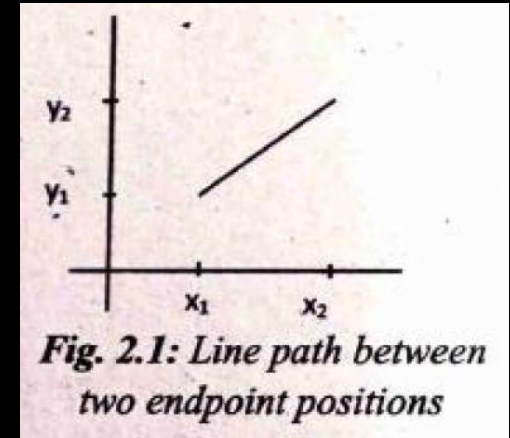
$$\therefore b = y - \frac{y_2 - y_1}{x_2 - x_1} x \text{ from above equation}$$

At any point (x_k, y_k)

$$y_k = mx_k + b \dots\dots\dots 1$$

At (x_{k+1}, y_{k+1}) ,

$$y_{k+1} = mx_{k+1} + b \dots\dots\dots 2$$



Line Scan Conversion

subtracting 1 from 2 we get-

$$y_{k+1} - y_k = m(x_{k+1} - x_k)$$

Here $(y_{k+1} - y_k)$ is increment in y as corresponding increment in x.

$$\therefore \Delta y = m \cdot \Delta x \dots\dots\dots 3$$

$$\text{or } m = \frac{\Delta y}{\Delta x} \dots\dots\dots 4$$

When the value of m is calculated from equation 4 there are three cases that arises:

Case I: $|m| < 1$

Case II: $|m| > 1$

Case III: $|m| = 1$

Digital Differential Analyzer (DDA)

DDA

- It is a scan conversion line algorithm based on calculation either Δx or Δy using equation $m = \Delta y / \Delta x$.
- We sample the line at unit interval in one direction (x if Δx is greater than Δy , otherwise in y direction) and determine the corresponding integer values nearest the line path for the other coordinate..

Algorithm

The equation of the line is given,

$$y = mx + b \dots\dots\dots(i)$$

$$m = (y_2 - y_1) / (x_2 - x_1) \dots\dots\dots(ii)$$

For any interval Δx , corresponding interval is given by $\Delta y = m \cdot \Delta x$.

Case I:

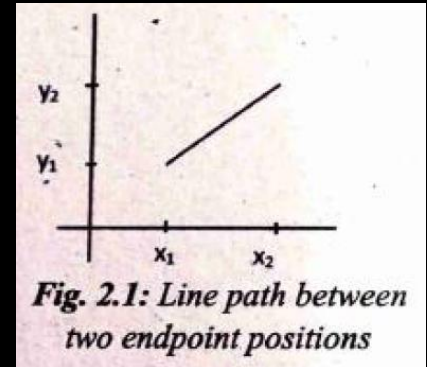
If $|m| \leq 1$, we sample at unit x interval i.e $\Delta x = 1$.

$$x_{k+1} = x_k + 1$$

Then we compute each successive y-values, by setting $\Delta y = m$

$$y_{k+1} = y_k + m$$

The calculated y value must be rounded to the nearest integer



DDA

Algorithm

Case II:

If $|m| > 1$, we sample at unit y-interval i.e $\Delta y = 1$ and compute each successive x-values.

$$y_{k+1} = y_k + 1$$

Therefore, $1 = m \cdot \Delta x$, and $\Delta x = 1/m$ (since, $m = \Delta y / \Delta x$ and $\Delta y = 1$).

$$x_{k+1} = x_k + 1/m$$

The above equations are under the assumption that lines are to be processed from left endpoints (x_k, y_k) to right endpoints (x_{k+1}, y_{k+1}) .



DDA

	Moving Left to Right		Moving Right to Left	
	Slope (m) < 1	Slope (m) > 1	Slope (m) < 1	Slope (m) > 1
Positive Slope Fig. Positive Slope	 $x_{k+1} = x_k + 1$ $y_{k+1} = y_k + m$	 $x_{k+1} = x_k + \frac{1}{m}$ $y_{k+1} = y_k + 1$	 $x_{k+1} = x_k - 1$ $y_{k+1} = y_k - m$	 $x_{k+1} = x_k - \frac{1}{m}$ $y_{k+1} = y_k - 1$
Negative Slope Fig. Negative Slope	 $x_{k+1} = x_k + 1$ $y_{k+1} = y_k - m$	 $x_{k+1} = x_k + \frac{1}{m}$ $y_{k+1} = y_k - 1$	 $x_{k+1} = x_k - 1$ $y_{k+1} = y_k + m$	 $x_{k+1} = x_k - \frac{1}{m}$ $y_{k+1} = y_k + 1$

DDA

Pros of DDA

1. It is simple to understand.
2. It is faster method than direct use of line equation $y = mx + c$. (Removes multiplication operation and only involve increments operation of x or y direction)
3. It requires no special skills for implementation

Cons of DDA

1. A floating-point addition is still needed in determining each successive point which is time consuming.
2. The accumulation of round off error in successive addition of the floating-point increments may cause the calculated pixel position to drift away from the true line path for long line segment.

Numerical on DDA

Numerical on DDA

Question 1:

Consider a line from (2,1) to (8,3). Using simple DDA algorithm, rasterize this line.



Consider a line from (2,1) to (8,3). Using simple DDA algorithm, rasterize this line.

Given,

Starting Point : $(x_1, y_1) = (2, 1)$

Ending Point: $(x_2, y_2) = (8, 3)$

Now,

$$\begin{aligned}\text{Slope } m &= (y_2 - y_1) / (x_2 - x_1) = (3 - 1) / (8 - 2) \\ &= (1/3) = 0.333\end{aligned}$$

X	Y	X-Plot	Y-Plot	(X,Y)
2	1	2	1	(2,1)
3 (2+1)	1.33 (1+0.33)	3	1	(3,1)
4 (3+1)	1.66 (1.33+0.33)	4	2	(4,2)
5	1.993	5	2	(5,2)
6	2.326	6	2	(6,2)
7	2.659	7	3	(7,3)
8	2.999	8	3	(8,3)

Since $|m| < 1$, From DDA algorithm we have

$$X_{k+1} = X_k + 1$$

$$Y_{k+1} = Y_k + m$$

Numerical on DDA

Question 2:

Digitized the line with end points $(0,0)$ and $(4,5)$ using DDA.

Digitized the line with end points (0,0) and (4,5) using DDA.

Given Starting Point : $(x_1, y_1) = (0,0)$ and Ending

Point: $(x_2, y_2) = (4,5)$

Now Slope $m = (y_2 - y_1) / (x_2 - x_1) = (5 - 0) / (4 - 0) = (5/4) =$

1.25

Since $|m| > 1$, From DDA algorithm we have

$$y_{k+1} = y_k + 1$$

$$x_{k+1} = x_k + (1/m)$$

X	Y	X-Plot	Y-Plot	(X,Y)
0	0	0	0	(0,0)
0.8	1	1	1	(1,1)
1.6	2	2	2	(2,2)
2.4	3	3	3	(3,3)
3.2	4	4	4	(4,4)
4	5	4	5	(4,5)

Numerical on DDA

Question 3:

Digitized the line with end points (3,7) and (8,3) using DDA

Digitized the line with end points (3,7) and (8,3) using DDA

Given Starting Point : $(x_1, y_1) = (3, 7)$ and Ending Point: (x_2, y_2)

$= (8, 3)$

Now,

Slope $m = (y_2 - y_1) / (x_2 - x_1) = (3 - 7) / (8 - 3) = (-4/5) = -0.8$

Since $|m| < 1$, From DDA algorithm we have

$$X_{k+1} = X_k + 1$$

$$Y_{k+1} = Y_k - m$$

X	Y	X-Plot	Y-Plot	(X,Y)
3	7	3	7	(3,7)
4	$7 - 0.8 = 6.2$	4	6	(4,6)
5	5.4	5	5	(5,5)
6	4.6	6	5	(6,5)
7	3.8	7	4	(7,4)
8	3	8	3	(8,3)

Bresenham's Line Algorithm (BLA)

Bresenham's Line Algorithm (BLA)

- The Bresenham Line Algorithm is an efficient method for drawing a line between two points in a discrete plane, such as a computer screen composed of pixels.
- The basic idea of the algorithm is to **determine the pixels that should be selected to approximate the straight line** between two given points.
- The algorithm **avoids floating-point arithmetic**, which was particularly beneficial in early computer systems where floating-point operations were more computationally expensive.
- The Bresenham algorithm is efficient because **it only involves integer addition and subtraction, avoiding the need for multiplication or division**. This makes it suitable for real-time graphics applications.

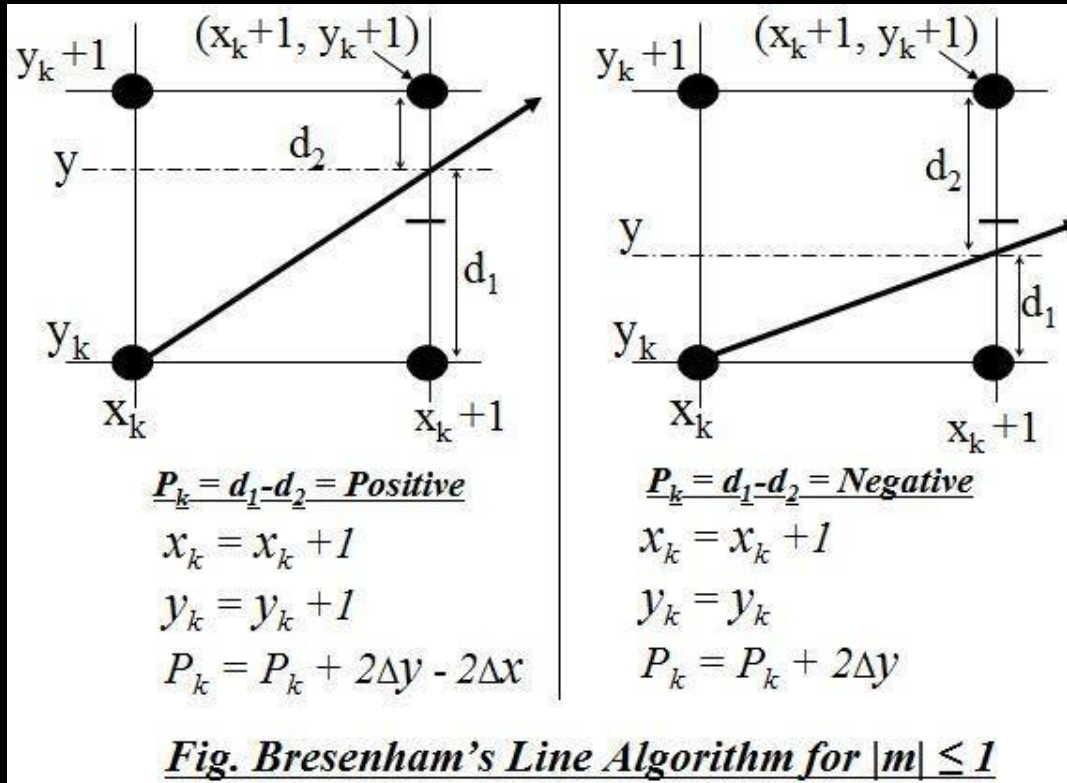
Bresenham's Line Algorithm (BLA)

The BLA is a more efficient method used to plot pixel position along a straight-line path.

Advantages of BLA over DDA

- In DDA algorithm each successive point is computed in *floating point*, so it requires *more time* and *more memory space*. While in BLA each successive point is calculated in *integer value or whole number*. So, it requires less time and less memory space.
- In DDA, since the calculated point value is floating point number, it should be rounded at the end of calculation but in BLA it does not need to round, so there is *no accumulation of rounding error*.
- Due to rounding error, the line drawn by DDA algorithm is not accurate, while in BLA line is accurate.
- DDA algorithm cannot be used in other application except line drawing, but BLA can be implemented in other application such as circle, ellipse and other curves.

BLA for slope +ve and $|m| \leq 1$



BLA for slope +ve and $|m| \leq 1$

Assume that (x_k, y_k) is pixel at kth step then next point to plot may be either $(x_k + 1, y_k)$ or $(x_k + 1, y_k + 1)$

$$y = m(x_k + 1) + b \text{ ----- 1}$$

At sampling position $x_k + 1$, we label vertical pixel separation from line path as d_1 and d_2 as in figure. The y-coordinate on the mathematical line path at pixel column $x_k + 1$ is $y = m(x_k + 1) + b$

$$\text{Then } d_1 = y - y_k$$

$$= m(x_k + 1) + b - y_k \text{ ----- 2}$$

$$d_2 = (y_k + 1) - y$$

$$= (y_k + 1) - m(x_k + 1) - b \text{ ----- 3}$$

$$\text{Now } d_1 - d_2 = [m(x_k + 1) + b - y_k] - [(y_k + 1) - m(x_k + 1) - b]$$

$$= m(x_k + 1) + b - y_k - (y_k + 1) + m(x_k + 1) + b$$

$$= 2m(x_k + 1) - (y_k + 1) - y_k + 2b$$

$$= 2m(x_k + 1) - 2y_k + 2b - 1$$

BLA for slope +ve and $|m| \leq 1$

A decision parameter p_k for the k^{th} step in the line algorithm can be obtained by substituting $m = \frac{\Delta y}{\Delta x}$ in above eqⁿ and defining

$$\begin{aligned}
 p_k &= \Delta x(d_1 - d_2) \\
 &= \Delta x \left[2 \frac{\Delta y}{\Delta x} (x_k + 1) - 2y_k + 2b - 1 \right] \\
 &= 2\Delta y.x_k - 2\Delta x.y_k + 2\Delta y + \Delta x(2b - 1) \\
 &= 2\Delta y.x_k - 2\Delta x.y_k + c \quad \text{----- 4}
 \end{aligned}$$

Where the constant $c = 2\Delta y + \Delta x(2b - 1)$ which is independent.

If decision parameter p_k is negative i.e. $d_1 < d_2$, pixel at y_k is closer to the line path than pixel at y_{k+1} . In this case we plot lower pixel $(x_k + 1, y_k)$ otherwise plot upper pixel (x_{k+1}, y_{k+1}) .

BIA for slope +ve and $|m| \leq 1$

At step $k+1$, p_{k+1} is evaluated as:

$$p_{k+1} = 2\Delta y \cdot x_{k+1} - 2\Delta x y_{k+1} + c$$

We get,

$$p_{k+1} - p_k = 2\Delta y(x_{k+1} - x_k) - 2\Delta x(y_{k+1} - y_k) \text{ ----- 5}$$

From eq. 5, we have:

Since we are sampling at X-direction, we have $x_{k+1} = x_k + 1$

$$\therefore p_{k+1} = p_k + 2\Delta y - 2\Delta x(y_{k+1} - y_k) \text{ ----- 6}$$

The term y_k is either 0 or 1 depending upon the sign of p_k .

So, From eq. 6, we have:

if p_k is negative

$$y_{k+1} = y_k \text{ and } P_{k+1} = p_k + 2\Delta y$$

otherwise

$$y_{k+1} = y_k + 1, \text{ then } p_{k+1} = p_k + 2\Delta y - 2\Delta x$$

BLA for slope +ve and $|m| \leq 1$

Q. If the starting point is (x_0, y_0) , then what will be the value of initial decision parameter (P_0) when $|m| \leq 1$?

We have $P_k = \Delta x(d_1 - d_2)$

$$P_k = \Delta x (2m(x_k + 1) - 2y_k + 2b - 1)$$

$$P_k = \Delta x (2mx_k + 2m + 2b - 2y_k - 1)$$

$$P_k = \Delta x \{2(mx_k + b - y_k) + 2m - 1\}$$

The line $y = mx + b$ must pass through the initial point (x_0, y_0) , $(mx_k + b - y_k) = 0$,

we have

$$P_0 = \Delta x (2m - 1)$$

$$P_0 = \Delta x (2 \Delta y / \Delta x - 1)$$

$$P_0 = 2\Delta y - \Delta x, \text{ which is the initial decision parameter.}$$

Numerical on BLA

Question 1:

Digitized the line with end points (20,10) and (30,18) using BLA.

Digitized the line with end points (20,10) and (30,18) using BLA

The initial decision parameter $(P_0) = 2\Delta y - \Delta x = 2*8 - 10 = 6$

Since, for the Bresenham's Line drawing Algorithm of slope, $|m| \leq 1$, we have

If $P_k < 0$ (i.e. $d_1 - d_2$ is Negative)
then,

$$X_{k+1} = X_k + 1$$

$$Y_{k+1} = Y_k$$

$$P_k = P_k + 2 \Delta y$$

If $P_k \geq 0$

then,

$$X_{k+1} = X_k + 1$$

$$Y_{k+1} = Y_k + 1$$

$$P_k = P_k + 2 \Delta y - 2 \Delta x$$

(20,

10)

k	P_k	X_{k+1}	Y_{k+1}	(X_{k+1}, Y_{k+1})
0.	6	$20+1 = 21$	11	(21, 11)
1.	$6 + 2*8 - 2*10 = 2$	$21+1 = 22$	12	(22, 12)
2.	$2 + 2*8 - 2*10 = -2$	$22+1 = 23$	12	(23, 12)
3.	$-2 + 2*8 = 14$	$23+1 = 24$	13	(24, 13)
4.	$14 + 2*8 - 2*10 = 10$	$24+1 = 25$	14	(25, 14)
5.	$10 + 2*8 - 2*10 = 6$	$25+1 = 26$	15	(26, 15)
6.	$6 + 2*8 - 2*10 = 2$	$26+1 = 27$	16	(27, 16)
7.	$2 + 2*8 - 2*10 = -2$	$27+1 = 28$	16	(28, 16)
8.	$-2 + 2*8 = 14$	$28+1 = 29$	17	(29, 17)
9.	$14 + 2*8 - 2*10 = 10$	$29+1 = 30$	18	(30, 18)

BLA for slope +ve and $|m| > 1$

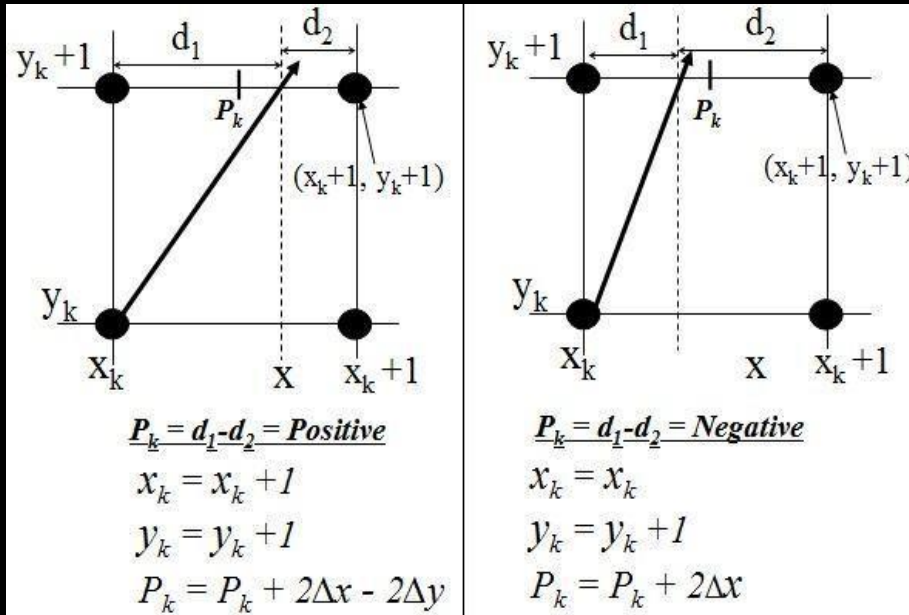


Fig. Bresenham's Line Algorithm for $|m| > 1$

Derive it yourself.

Numerical on BLA

Question 2:

Digitized the line with end points $(1,0)$ and $(3,3)$ using BLA.

Digitized the line with end points (1,0) and (3,3) using BLA.

Thus, the initial decision parameter

$$(P_0) = 2\Delta x - \Delta y = 2*2 - 3 = 1$$

we have

$$\Delta x = x_2 - x_1 = 3 - 1 = 2$$

$$\Delta y = y_2 - y_1 = 3 - 0 = 3$$

If $P_k < 0$

then,

$$X_{k+1} = X_k$$

$$Y_{k+1} = Y_k + 1$$

$$P_k = P_k + 2 \Delta x$$

If $P_k \geq 0$

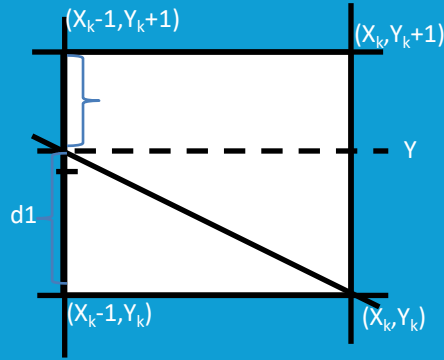
then,

$$X_{k+1} = X_k + 1$$

$$Y_{k+1} = Y_k$$

$$P_k = P_k + 2 \Delta x - 2 \Delta y$$

k	P_k	X_{k+1}	Y_{k+1}	(X_{k+1}, Y_{k+1})
0.	1	2	1	(2, 1)
1.	$1 + 2*2 - 2*3 = -1$	2	$1+1 = 2$	(2, 2)
2.	$-1 + 2*2 = 3$	3	$2+1 = 3$	(3, 3)



$d1 - d2 > 0$ (Positive)

$$X_{k+1} = X_k - 1$$

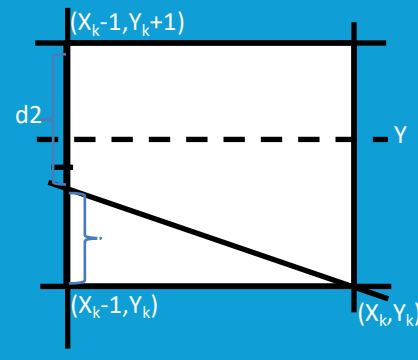
$$Y_{k+1} = Y_k + 1$$

If $P_k \geq 0$

$$X_{k+1} = X_k - 1$$

$$Y_{k+1} = Y_k + 1$$

$$P_k = P_k + 2 \Delta y - 2 \Delta x$$



$d1 - d2 < 0$ (Negative)

$$X_{k+1} = X_k - 1$$

$$Y_{k+1} = Y_k$$

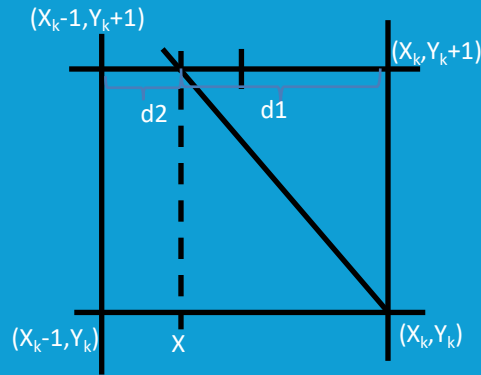
If $P_k < 0$

$$X_{k+1} = X_k - 1$$

$$Y_{k+1} = Y_k$$

$$P_k = P_k + 2 \Delta y$$

$$P_0 = 2 \Delta y - \Delta x$$



$d1-d2 > 0$ (Positive)

$$X_{k+1} = X_k - 1$$

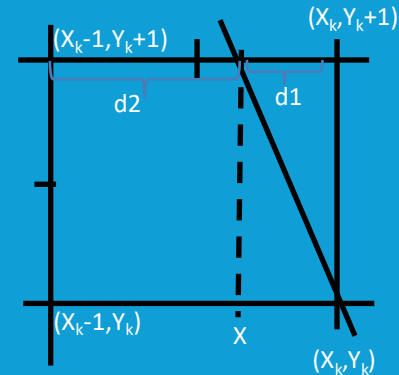
$$Y_{k+1} = Y_k + 1$$

If $P_k \geq 0$

$$X_{k+1} = X_k - 1$$

$$Y_{k+1} = Y_k + 1$$

$$P_k = P_k + 2\Delta x - 2\Delta y$$



$d1-d2 < 0$ (Negative)

$$X_{k+1} = X_k$$

$$Y_{k+1} = Y_k + 1$$

If $P_k < 0$

$$X_{k+1} = X_k$$

$$Y_{k+1} = Y_k + 1$$

$$P_k = P_k + 2\Delta x$$

$$P_0 = 2\Delta x - \Delta y$$

Solution:

Here, $(X_1, Y_1) = (6, 2)$ &
 $(X_2, Y_2) = (3, 10)$

$M = (10-2)/(3-6) = 8/-3$ –ve slope and $|M| > 1$

If $P_k < 0$

$$X_{k+1} = X_k$$

$$Y_{k+1} = Y_k + 1$$

$$P_k = P_k + 2 \Delta x$$

If $P_k \geq 0$

$$X_{k+1} = X_k - 1$$

$$Y_{k+1} = Y_k + 1$$

$$P_k = P_k + 2\Delta x - 2\Delta y$$

$$\Delta x = |3-6| = 3$$

$$\Delta y = |10-2| = 8$$

here,

$$\text{Initial } (P_0) = 2 \Delta x - \Delta y$$

$$= 2 \times 3 - 8$$

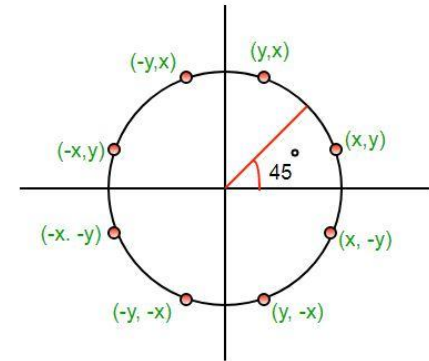
$$= -2$$

(6, 2)				
K	P_k	X_{k+1}	Y_{k+1}	(X_{k+1}, Y_{k+1})
0	-2	6	3	(6,3)
1	$= -2 + 2 \times 3 = 4$	5	4	(5,4)
2	$= 4 + 6 - 16 = -6$	5	5	(5,5)
3	$= -6 + 6 = 0$	4	6	(4,6)
4	$= 0 + 6 - 16 = -10$	4	7	(4,7)
5	$= -10 + 6 = -4$	4	8	(4,8)
6	$= -4 + 6 = 2$	3	9	(3,9)
7	$= -8$	3	10	(3,10)

Circle Algorithm

What is Circle ?

- Similarly to the case with lines, there is an incremental algorithm for drawing circles – the **mid-point circle algorithm**
- **Mid-point circle drawing algorithm** is an algorithm used to determine the points needed for rasterizing a circle.
- In the mid-point circle algorithm we use eight-way symmetry so only ever calculate the points for the top right eighth of a circle, and then use symmetry to get the rest of the points.

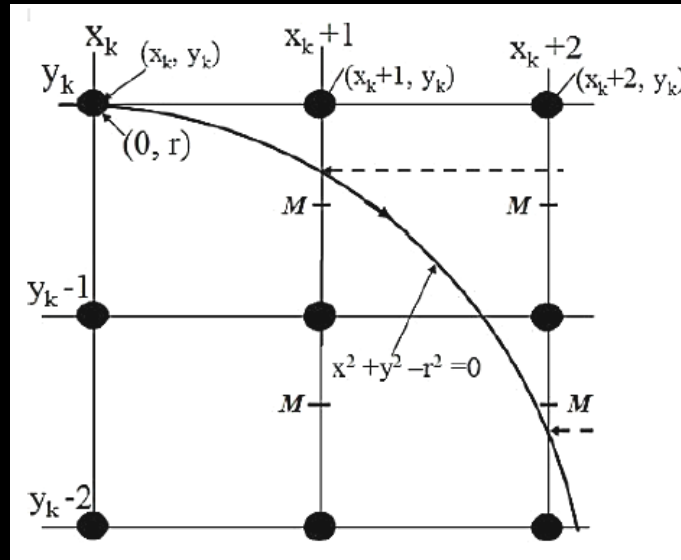


The equation of circle at (0,0) is

$$x^2 + y^2 = r^2 \dots\dots\dots (1)$$



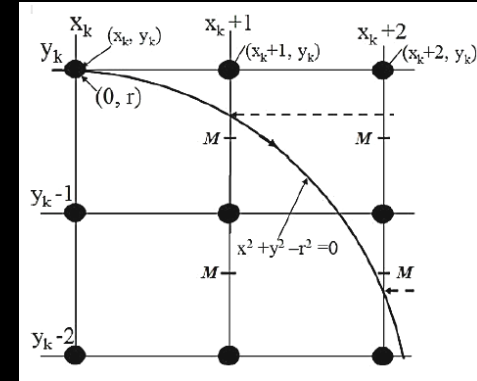
Mid Point Circle Algorithm





Mid Point Circle Algorithm

Suppose, (x_k, y_k) is the pixel plotted, then the next pixel (x_{k+1}, y_{k+1}) will be either (x_k+1, y_k) or (x_k+1, y_k-1) .
Here,
the mid-point $(m) = (x_k+1, y_k-1/2)$.



Now, decision parameter $(P_k) = f_{\text{circle}}(x_k+1, y_k-1/2) = (x_k+1)^2 + (y_k-1/2)^2 - r^2$, is the circle function evaluated at midpoint.

Also, the next pixel to plot will either be $(x_{k+1} + 1, y_{k+1})$ or $(x_{k+1} + 1, y_{k+1} - 1)$

Also, the midpoint here is $(x_{k+1} + 1, y_{k+1} - 1/2)$

And the next decision parameter

$$P_{k+1} = f_{\text{circle}}(x_{k+1} + 1, y_{k+1} - 1/2)$$

$$P_{k+1} = (x_{k+1} + 1)^2 + (y_{k+1} - 1/2)^2 - r^2$$

$$P_{k+1} = \{(x_k + 1) + 1\}^2 + (y_{k+1} - 1/2)^2 - r^2$$

Mid Point Circle Algorithm

$$P_{k+1} = (x_k + 1)^2 + 2(x_k + 1) + 1 + (y_{k+1} - 1/2)^2 - r^2$$

Now, subtracting P_{k+1} and P_k , we have

$$\begin{aligned} P_{k+1} - P_k &= \{(x_k + 1)^2 + 2(x_k + 1) + 1 + (y_{k+1} - 1/2)^2 - r^2\} - \{(x_k)^2 + (y_k - 1/2)^2 - r^2\} \\ &= 2(x_k + 1) + 1 + y_{k+1}^2 - y_k^2 + 1/2^2 - y_k^2 + y_k - 1/2^2 \\ &= 2(x_k + 1) + (y_{k+1}^2 - y_k^2) - (y_{k+1} - y_k) + 1 \end{aligned}$$

If $P_k < 0$, the midpoint is inside the circle and the pixel on the scan line y_k closer to the circle boundary. Otherwise, the midpoint is outside or on the circle boundary, and we select the pixel on scan line $y_k + 1$.

Case $P_k < 0$:

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k$$

$$P_{k+1} = P_k + 2(x_k + 1) + 1$$

$$P_{k+1} = P_k + 2x_{k+1} + 1$$

Mid Point Circle Algorithm

Case $P_k \geq 0$:

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k - 1$$

$$\begin{aligned} P_{k+1} &= P_k + 2(x_k + 1) + [(y_k - 1)^2 - y_k^2] - (y_k - 1 - y_k) + 1 \\ &= P_k + 2(x_k + 1) + [y_k^2 - 2y_k + 1 - y_k^2] - (y_k - 1 - y_k) + 1 \\ &= P_k + 2(x_k + 1) - 2y_k + 1 + 1 + 1 \\ &= P_k + 2(x_k + 1) - 2y_k + 2 + 1 \\ &= P_k + 2(x_k + 1) - 2(y_k - 1) + 1 \\ &= P_k + 2x_{k+1} - 2y_{k+1} + 1 \end{aligned}$$

Q. Calculate the initial decision parameter (P_0) for circle

The initial decision parameter is obtained by evaluating the circle function at the start position $(x_0, y_0) = (0, r)$.

Here, the next pixel will either be $(1, r)$ or $(1, r-1)$ where the midpoint is $(1, r - \frac{1}{2})$. Thus, the initial decision parameter is given by:

$$\begin{aligned} P_0 &= f_{\text{circle}}(1, r - \frac{1}{2}) \\ &= 1^2 + (r - \frac{1}{2})^2 - r^2 \\ &= 5/4 - r \\ &\approx 1 - r \end{aligned}$$

Thus, $P_0 = 1 - r$

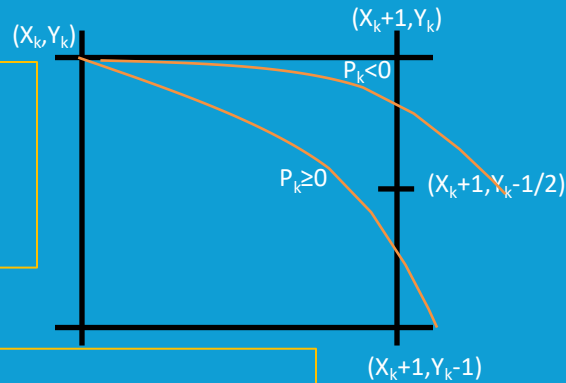
Case 1 : $P_k < 0$

$$X_{k+1} = X_k + 1$$

$$Y_{k+1} = Y_k$$

$$P_{k+1} = P_k + 2(X_{k+1}) + 1$$

$$\mathbf{P_{k+1} = P_k + 2X_{k+1} + 1}$$



Case 2: $P_k \geq 0$

$$X_{k+1} = X_k + 1$$

$$Y_{k+1} = Y_k - 1$$

$$\begin{aligned} P_{k+1} &= P_k + 2(X_{k+1}) + [(Y_k - 1)^2 - Y_k^2] - (Y_k - 1 - Y_k) + 1 \\ &= P_k + 2(X_{k+1}) + [(Y_k^2 - 2Y_k + 1 - Y_k^2)] - (Y_k - 1 - Y_k) + 1 \\ &= P_k + 2(X_{k+1}) - 2Y_k + 1 + 1 + 1 \\ &= P_k + 2(X_{k+1}) - 2(Y_k + 1) + 1 \end{aligned}$$

$$\mathbf{P_{k+1} = P_k + 2X_{k+1} - 2Y_{k+1} + 1}$$

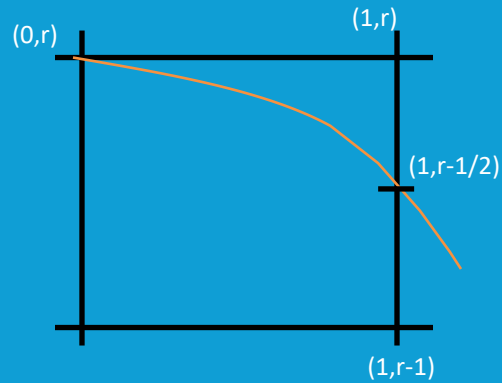
i.e.,
 $(x_0, y_0) = (0, r)$

Here ,
 $F_{\text{circle}}(1, r-1/2) = P_0$

Now,

$$\begin{aligned} P_0 &= 1^2 + (r-1/2)^2 - r^2 \\ &= 1 + r^2 - r + 1/4 - r^2 \\ &= 5/4 - r \\ P_0 &\approx 1-r \end{aligned}$$

$$P_0 \approx 1-r$$



Midpoint Circle Algorithm

1. Input radius r and circle center (x_c, y_c) , then set the coordinates for the first point on the circumference of a circle centered on the origin as $(x_0, y_0) = (0, r)$.
2. Calculate the initial value of the decision parameter as $p_0 = 5/4 - r$ (1 - r if an integer)
3. At each x_k , from $k=0$, perform the following test:
 - if $p_k < 0$, next point to plot along the circle centered on $(0,0)$ is $(x_k + 1, y_k)$ and $p_{k+1} = p_k + 2x_{k+1} + 1$
 - otherwise, next point to plot is $(x_k + 1, y_k - 1)$
 and $p_{k+1} = p_k + 2x_{k+1} + 1 - 2y_{k+1}$
 where $2x_{k+1} = 2x_k + 2$, and $2y_{k+1} = 2y_k - 2$

Midpoint Circle Algorithm

4. Determine symmetry points in the other seven octants.
5. Move each calculated pixel position (x, y) onto the circular path centered at (x_c, y_c) and plot the coordinate values:
 $x = x + x_c, y = y + y_c$
6. Repeat steps 3 through 5 until $x \geq y$.

Numerical on Mid point Circle Algorithm

Question 1:

Digitize a circle with radius 10 at center (0,0)

Question 1:

Digitize a circle with radius 10 at center (0,0)

Case 1 : $P_k < 0$

$$X_{k+1} = X_k + 1$$

$$Y_{k+1} = Y_k$$

$$P_{k+1} = P_k + 2X_{k+1} + 1$$

Case 2: $P_k \geq 0$

$$X_{k+1} = X_k$$

$$Y_{k+1} = Y_k + 1$$

$$P_{k+1} = P_k - 2Y_{k+1} + 1$$

Here, the initial decision parameter (P_0) = $1 - r = 1 - 10 = -9$

Since, for the Midpoint Circle Algorithm of initial point(0, r)

& center at origin (0, 0) rotating at clockwise direction, we

have

Initial point (x_0, y_0) = (0,10)

$$P_0 = 1 - r = 1 - 10 = -9$$

Case 1 : $P_k < 0$

$$X_{k+1} = X_k + 1$$

$$Y_{k+1} = Y_k$$

$$P_{k+1} = P_k + 2X_{k+1} + 1$$

Case 2: $P_k \geq 0$

$$X_{k+1} = X_k$$

$$Y_{k+1} = Y_k - 1$$

$$P_{k+1} = P_k - 2Y_{k+1} + 1$$

Solution

Here, the initial decision parameter (P_0) = $1 - r = 1 - 10 = -9$

Since, for the Midpoint Circle Algorithm of initial point(0, r)

& center at origin (0, 0) rotating at clockwise direction, we

have

Initial point (x_0, y_0) = (0,10)

$$P_0 = 1 - r = 1 - 10 = -9$$

Thus,

(0, 10)

K	P_k	X_{k+1}	Y_{k+1}	(X_{k+1}, Y_{k+1})
0	-9	$0+1=1$	10	(1,10)
1	$= -9 + 2*1 +1= -6$	$1+2=2$	10	(2, 10)
2	$= -6 + 2*2 +1= -1$	$2+1=3$	10	(3, 10)
3	$= -1 + 2*3 +1= 6$	$3+1= 4$	$10-1=9$	(4, 9)
4	$= 6 + 2*4 - 2*9 +1= -3$	$4+1= 5$	9	(5, 9)
5	$= -3 + 2*5 +1= 8$	$5+1= 6$	$9-1=8$	(6, 8)
6	$= 8 + 2*6 - 2*8 +1= 5$	$6+1= 7$	$8-1=7$	(7, 7)
7	$= 5 + 2*7 - 2*7 +1= 6$	$7+1= 8$	$7-1=6$	(8, 6)

Numerical on Mid point Circle Algorithm

Question 2:

Digitize a circle with radius 9 at center (6,7)

Here, the initial decision parameter (P_0)

$$P_0 = 1 - r = 1 - 9 = -8$$

Since, for the Midpoint Circle Algorithm of starting point (0, r) & center at origin (0, 0) rotating at clockwise direction, we have

If $P < 0$

Plot ($x_k + 1, y_k$)

$$P_{k+1} = P_k + 2x_{k+1} + 1$$

Else ($P \geq 0$)

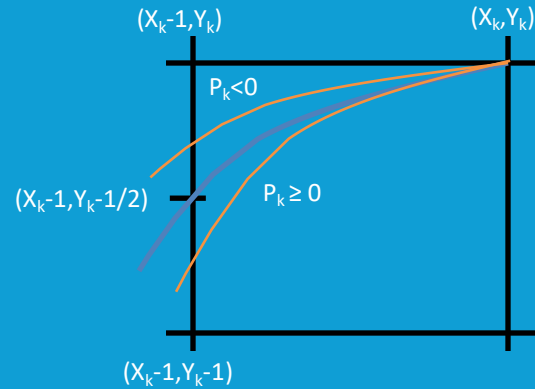
Plot ($x_k + 1, y_k - 1$)

$$P_{k+1} = P_k + 2x_{k+1} - 2y_{k+1} + 1$$

k	P_k	X_{k+1}	Y_k + 1	(X_{k+1}, Y_{k+1}) At (0, 0)	(X_{k+1}, Y_{k+1}) At (6, 7)
0.	-8	$0+1 = 1$	9	(1, 9)	$(1+6, 9+7) = (7, 16)$
1.	$= -8 + 2*1 + 1 = -5$	$1+1 = 2$	9	(2, 9)	$(2+6, 9+7) = (8, 16)$
2.	$= -5 + 2*2 + 1 = 0$	$2+1 = 3$	8	(3, 8)	$(3+6, 8+7) = (9, 15)$
3.	$= 0 + 2*3 - 2*8 + 1 = -9$	$3+1 = 4$	8	(4, 8)	$(4+6, 8+7) = (10, 15)$
4.	$= -9 + 2*4 + 1 = 0$	$4+1 = 5$	7	(5, 7)	$(5+6, 8+7) = (11, 14)$
5.	$= 0 + 2*5 - 2*7 + 1 = -3$	$5+1 = 6$	7	(6, 7)	$(6+6, 7+7) = (12, 14)$
6.	$= -3 + 2*6 + 1 = 10$	$6+1 = 7$	6	(7, 6)	$(7+6, 6+7) = (13, 13)$

Mid Point Circle Algorithm

Anti-Clockwise direction



$$\begin{aligned} P_k &< 0 \\ X_{K+1} &= X_k - 1 \\ Y_{K+1} &= Y_k \end{aligned}$$

$$\begin{aligned} P_k &\geq 0 \\ X_{K+1} &= X_k - 1 \\ Y_{K+1} &= Y_k - 1 \end{aligned}$$

Here,

$$\begin{aligned}\text{Decision parameter}(P_k) &= F_{\text{circle}}(X_k - 1, Y_k - 1/2) \\ &= (X_k - 1)^2 + (Y_k - 1/2)^2 - r^2\end{aligned}$$

Then, $K+1^{\text{th}}$ term is,

$$P_{k+1} = (X_{k+1} - 1)^2 + (Y_{k+1} - 1/2)^2 - r^2$$

Now,

$$\begin{aligned}P_{k+1} - P_k &= (X_{k+1} - 1)^2 + (Y_{k+1} - 1/2)^2 - r^2 - \{(X_k - 1)^2 + (Y_k - 1/2)^2 - r^2\} \\ &= -2(X_k - 1) + (Y_{k+1}^2 - Y_k^2) - (Y_{k+1} - Y_k) + 1\end{aligned}$$

[$\therefore X_{k+1} = X_k - 1$ in both condition]

$$P_{k+1} = P_k - 2X_{k+1} + (Y_{k+1}^2 - Y_k^2) - (Y_{k+1} - Y_k) + 1$$

Case 1 : $P_k < 0$

$$X_{k+1} = X_k - 1$$

$$Y_{k+1} = Y_k$$

$$P_{k+1} = P_k - 2X_{k+1} + 1$$

Case 2: $P_k \geq 0$

$$X_{k+1} = X_k - 1$$

$$Y_{k+1} = Y_k + 1$$

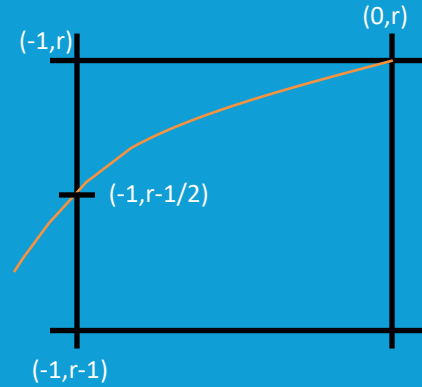
$$P_{k+1} = P_k - 2X_{k+1} - 2Y_{k+1} + 1$$

Initial decision parameter

i.e.,
 $(x_0, y_0) = (0, r)$

Here ,
 $F_{\text{circle}}(-1, r-1/2) = P_0$

Now,
$$P_0 = (-1)^2 + (r-1/2)^2 - r^2$$
$$= 1 + r^2 - r + 1/4 - r^2$$
$$= 5/4 - r$$
$$P_0 \approx 1-r$$



$$P_0 \approx 1-r$$

Mid point Ellipse Algorithm

Mid point Ellipse Algorithm

Our approach here is similar to that used in displaying a raster circle but the ellipse has 4-way symmetry. The midpoint ellipse method is applied throughout the first quadrant in two parts or region as shown in figure.

The region-1 just behaves as the circular property and the region-2 is slightly straight curve.

We have,

The equation of ellipse, whose centre at (0, 0) is

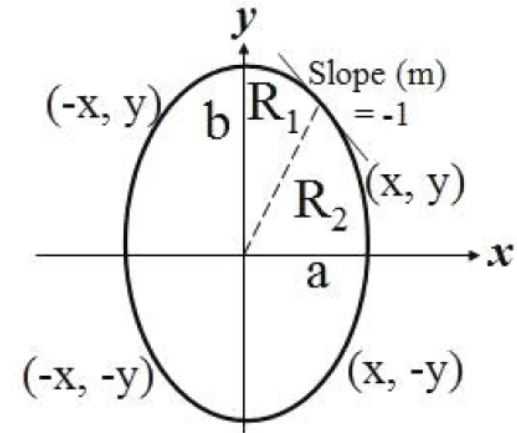
$$x^2 / a^2 + y^2 / b^2 = 1$$

Hence, we define the ellipse function for centre at origin (0, 0) as:

$$F_{\text{ellipse}}(x, y) = x^2 b^2 + y^2 a^2 - a^2 / b^2$$

This has the following properties:

$$F_{\text{ellipse}}(x, y) \begin{cases} < 0, & \text{if } (x, y) \text{ is inside the ellipse boundary} \\ = 0, & \text{if } (x, y) \text{ is on the ellipse boundary} \\ > 0, & \text{if } (x, y) \text{ is outside the ellipse boundary} \end{cases}$$



Mid point Ellipse Algorithm

Starting at $(0, b)$, we take unit steps in the x direction until we reach the boundary between region 1 and region 2. Then we switch to unit steps in the y direction over the remainder of the curve in the first quadrant. At each step, we need to test the value of the slope of the curve. The ellipse slope is calculated by differentiating the ellipse function as:

$$2xb^2 + 2ya^2 \cdot dy/dx = 0$$

Or

$$dy/dx = -2xb^2 / 2ya^2$$

At the boundary between region 1 and region 2, $dy/dx = -1$ and $2xb^2 = 2ya^2$. Therefore, we move out of region 1 whenever $2xb^2 \geq 2ya^2$

For Region – 1: Condition ($2xb^2 < 2ya^2$)

Assuming that the position (x_k, y_k) has been plotted, we determine next position (x_{k+1}, y_{k+1}) as either (x_{k+1}, y_k) or (x_{k+1}, y_{k-1}) by evaluating the decision parameter $P1_k$ as:

$$P1_k = F_{\text{ellipse}}(x_k + 1, y_k - \frac{1}{2})$$

$$= b^2(x_k + 1)^2 + a^2(y_k - \frac{1}{2})^2 - a^2b^2$$

Mid point Ellipse Algorithm

At next sampling position, the decision parameter will be

$$\begin{aligned}
 P1_{k+1} &= F_{\text{ellipse}}(x_{k+1} + 1, y_{k+1} - \frac{1}{2}) \\
 &= b^2(x_{k+1} + 1)^2 + a^2(y_{k+1} - \frac{1}{2})^2 - a^2b^2 \\
 &= b^2\{(x_k + 1) + 1\}^2 + a^2(y_{k+1} - \frac{1}{2})^2 - a^2b^2 \\
 &= b^2\{(x_k + 1)^2 + 2(x_k + 1) + 1\} + a^2(y_{k+1} - \frac{1}{2})^2 - a^2b^2
 \end{aligned}$$

$$\begin{aligned}
 \text{Now, } P1_{k+1} - P1_k &= (b^2\{(x_k + 1)^2 + 2(x_k + 1) + 1\} + a^2(y_{k+1} - \frac{1}{2})^2 - a^2b^2) - (b^2(x_k + 1)^2 + a^2(y_k - \frac{1}{2})^2 - a^2b^2) \\
 &= (b^2(x_k + 1)^2 + 2b^2(x_k + 1) + b^2 + a^2(y_{k+1})^2 - a^2y_{k+1} + a^2\frac{1}{4} - a^2b^2) - (b^2(x_k + 1)^2 + a^2y_k^2 - a^2y_k + a^2\frac{1}{4} - a^2b^2) \\
 &= 2b^2(x_k + 1) + b^2 + a^2\{(y_{k+1})^2 - y_k^2\} - (y_{k+1} - y_k)
 \end{aligned}$$

If $P1_k < 0$, the midpoint is inside the ellipse and the pixel on the scan line y_k closer to the ellipse boundary. Otherwise, the midpoint is outside or on the ellipse boundary, and we select the pixel on scan line y_{k+1} .

Case: $P1_k < 0$

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k$$



Mid point Ellipse Algorithm

$$P1_{k+1} = P1_k + 2b^2(x_{k+1}) + b^2$$

Case I: $P1_k \geq 0$

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k - 1$$

$$\begin{aligned} P1_{k+1} &= P1_k + 2b^2(x_k + 1) + b^2 + a^2\{(y_{k+1})^2 - y_k^2 - (y_{k+1} - y_k)\} \\ &= P1_k + 2b^2(x_k + 1) + b^2 + a^2\{(y_k - 1)^2 - y_k^2 - a^2(y_k - 1 - y_k)\} \\ &= P1_k + 2b^2(x_k + 1) + a^2[(y_k^2 - 2y_k + 1 - y_k^2)] - a^2(y_k - 1 - y_k) + b^2 \\ &= P1_k + 2b^2(x_k + 1) - 2a^2y_k + a^2 + a^2 + b^2 \\ &= P1_k + 2b^2(x_k + 1) - 2a^2(y_k - 1) + b^2 \\ &= P1_k + 2b^2x_{k+1} - 2a^2y_{k+1} + b^2 \end{aligned}$$

Initial decision parameter ($P1_0$) for region-1 of ellipse

The initial decision parameter is obtained by evaluating the ellipse function at the start position $(x_0, y_0) = (0, b)$.

Here, the next pixel will either be $(1, b)$ or $(1, b-1)$ where the midpoint is $(1, b - \frac{1}{2})$. Thus, the initial decision parameter is given by:

$$\begin{aligned} P1_0 &= F_{\text{ellipse}}(1, b - \frac{1}{2}) \\ &= b^2 + a^2(b - \frac{1}{2})^2 - a^2b^2 \\ &= b^2 - a^2b + a^2/4 \end{aligned}$$

Mid point Ellipse Algorithm

For Region – 2: Condition ($2xb^2 \geq 2ya^2$)

Assuming that the position (x_k, y_k) has been plotted, we determine next position (x_{k+1}, y_{k+1}) as either (x_{k+1}, y_{k+1}) or (x_k, y_{k-1}) by evaluating the decision parameter

$P2_k$ as:

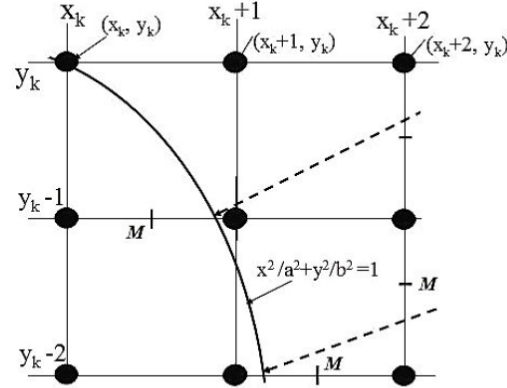
$$P2_k = F_{\text{ellipse}}(x_k + \frac{1}{2}, y_k - 1) \\ = b^2(x_k + \frac{1}{2})^2 + a^2(y_k - 1)^2 - a^2b^2$$

At next sampling position, the decision parameter will be

$$P2_{k+1} = F_{\text{ellipse}}(x_{k+1} + \frac{1}{2}, y_{k+1} - 1) \\ = b^2(x_{k+1} + \frac{1}{2})^2 + a^2(y_{k+1} - 1)^2 - a^2b^2$$

Now, subtracting $P2_k$ from $P2_{k+1}$, we have

$$\begin{aligned} P2_{k+1} - P2_k &= \{b^2(x_{k+1} + \frac{1}{2})^2 + a^2(y_{k+1}-1)^2 - a^2b^2\} - \{b^2(x_k + \frac{1}{2})^2 + a^2(y_k-1)^2 - a^2b^2\} \\ &= \{b^2(x_{k+1} + \frac{1}{2})^2 + a^2(y_{k+1}-1)^2 - a^2b^2 - b^2(x_k + \frac{1}{2})^2 - a^2(y_k-1)^2 + a^2b^2\} \\ &= \{b^2(x_{k+1} + \frac{1}{2})^2 + a^2(y_{k+1}-1)^2 - b^2(x_k + \frac{1}{2})^2 - a^2(y_k-1)^2\} \\ &= b^2(x_{k+1} + \frac{1}{2})^2 + a^2\{(y_k-1)-1\}^2 - b^2(x_k + \frac{1}{2})^2 - a^2(y_k-1)^2 \\ &= b^2x_{k+1}^2 + b^2x_{k+1} + b^2/4 + a^2(y_k-1)^2 - 2a^2(y_k-1) + a^2 - b^2x_k^2 - b^2x_k - b^2/4 - a^2(y_k-1)^2 \\ &= a^2 + b^2(x_{k+1}^2 - x_k^2) - 2a^2(y_k-1) + b^2(x_{k+1} - x_k) \end{aligned}$$



Mid point Ellipse Algorithm

If $P2_k < 0$, the midpoint is inside the ellipse and the pixel on the scan line x_k is closer to the ellipse boundary. Otherwise, the midpoint is outside or on the ellipse boundary, and we select the pixel on scan line x_{k+1} .

Case : $P2_k \leq 0$

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k - 1$$

$$\begin{aligned} P2_{k+1} &= P2_k + b^2 [(x_{k+1}^2 - x_k^2) + (x_{k+1} - x_k)] - 2a^2(y_k - 1) + a^2 \\ &= P2_k + b^2 [(x_k^2 + 2x_k + 1 - x_k^2)] + (x_k + 1 - x_k) - 2a^2(y_k - 1) + a^2 \\ &= P2_k + b^2 [2x_k + 1] - 2a^2(y_k - 1) + a^2 \\ &= P2_k + 2b^2(x_k + 1) - 2a^2(y_k - 1) + a^2 \\ &= P2_k + 2b^2x_{k+1} - 2a^2y_{k+1} + a^2 \end{aligned}$$

Case $P2_k > 0$

$$x_{k+1} = x_k$$

$$y_{k+1} = y_k - 1$$

$$\begin{aligned} P2_{k+1} &= P2_k + b^2 [(x_{k+1}^2 - x_k^2) + (x_{k+1} - x_k)] - 2a^2(y_k - 1) + a^2 \\ &= P2_k - 2a^2(y_k - 1) + a^2 \\ &= P2_k - 2a^2y_{k+1} + a^2 \end{aligned}$$

Initial decision parameter ($P2_0$) for region-2 of ellipse

When we enter region 2, the initial position (x_0, y_0) is taken as the last position selected in region 1 and the initial decision parameter in region 2 is given by:

$$\begin{aligned} P2_0 &= F_{\text{ellipse}}(x_0 + 1/2, y_0 - 1) \\ &= b^2(x_0 + 1/2)^2 + a^2(y_0 - 1)^2 - a^2b^2 \end{aligned}$$

Mid point Ellipse Algorithm

Question: Given input ellipse parameters $r_x = a = 8$ and $r_y = b = 6$, we illustrate the steps in the midpoint ellipse algorithm by determining raster positions along the ellipse path in the first quadrant

Given $a = 8$ and $b = 6$

For Region 1

The initial point for the ellipse on the origin $(x_0, y_0) = (0, b) = (0, 6)$

Calculation the initial decision parameter

$$\begin{aligned}
 P1_0 &= b^2 - a^2 x_0 + a^2 y_0 / 4 \\
 &= 6^2 - 6 * (8)^2 + (8^2 / 4) \\
 &= -332
 \end{aligned}$$

From midpoint algorithm, for Region 1 we know,

If $(P1_k < 0)$ then

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k$$

$$P1_{k+1} = P1_k + 2b^2 x_{k+1} + b^2$$

Else (i.e. $P1_k \geq 0$)

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k - 1$$

$$P1_{k+1} = P1_k + 2b^2 x_{k+1} - 2a^2 y_{k+1} + b^2$$

$$\text{And stop when } 2x_{k+1}b^2 \geq 2y_{k+1}a^2$$

Mid point Ellipse Algorithm

Now successive decision parameter values and positions along the ellipse path are calculated using midpoint method as

K	$P1_k$	(X_{k+1}, Y_{k+1})	$2b^2(x_{k+1})$	$2a^2 y_{k+1}$
0	-332	(1, 6)	72	768
1	-224	(2, 6)	144	768
2	-44	(3, 6)	216	768
3	208	(4, 5)	288	640
4	-108	(5, 5)	360	640
5	288	(6, 4)	432	512
6	244	(7, 3)	504	384

Since $2x_{k+1}b^2 \geq 2y_{k+1}a^2$ so we stop here for region 1.

Note: (7, 3) is the starting point for region 2]

Mid point Ellipse Algorithm

Now for Region-2,

From midpoint algorithm, for Region 2 we know,

Initial Decision Parameter:

$$P_{2_0} = b^2 (x_0 + 1/2)^2 + a^2 (y_0 - 1)^2 - a^2 b^2$$

If($P_k < 0$) then

$$X_{k+1} = X_k + 1$$

$$Y_{k+1} = Y_k - 1$$

$$P_{2_{k+1}} = P_{2_k} + 2b^2 x_{k+1} - 2a^2 y_{k+1} + a^2$$

Else (i.e. $P_k \geq 0$)

$$X_{k+1} = X_k$$

$$Y_{k+1} = Y_k - 1$$

$$P_{2_{k+1}} = P_{2_k} - 2a^2 y_{k+1} + a^2$$

So, the initial point for region 2 is $(x_0, y_0) = (7, 3)$ and the initial decision parameter is

$$\begin{aligned} P_{2_0} &= b^2 (x_0 + 1/2)^2 + a^2 (y_0 - 1)^2 - a^2 b^2 \\ &= 36 * (7 + 0.5)^2 + 64 * (3-1)^2 - 64 * 36 \\ &= -23 \end{aligned}$$

Mid point Ellipse Algorithm

The remaining pixels in region two for first quadrant are then calculated as

K	$P2_k$	(X_{k+1}, Y_{k+1})
0	-23	(8,2)
1	361	(8,1)
2	297	(8,0)

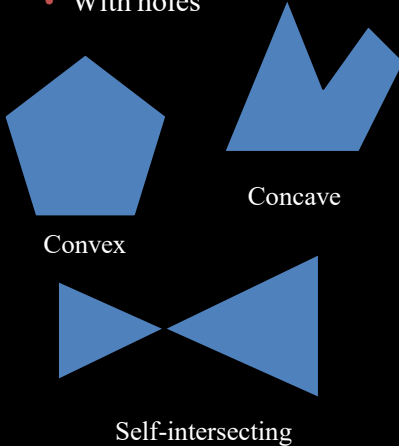
So, remaining points in other quadrants can be calculated using the symmetry property of ellipse.

Fill Area Algorithms

Polygon Fill Algorithm

- **Different types of Polygons**

- Simple Convex
- Simple Concave
- Non-simple : self-intersecting
- With holes



Area Fill Algorithm

- An area fill algorithm is a computer graphics algorithm used to fill a closed region or area with a specified color or pattern.
- This is often employed in the rendering of images and graphics on a computer screen.
- There are several algorithms for area filling, and the choice of algorithm depends on factors such as efficiency, simplicity, and the specific requirements of the application.
- The ***algorithm makes the following assumptions***
 - one interior pixel is known, and
 - pixels in boundary are known.

Scan Line Fill Algorithm

Area Fill Algorithm: Scan Line Fill Algorithm

Description: This algorithm works by scanning the image or polygon line by line, determining the intersections of the scan lines with the edges of the polygon, and filling the spaces between these intersections.

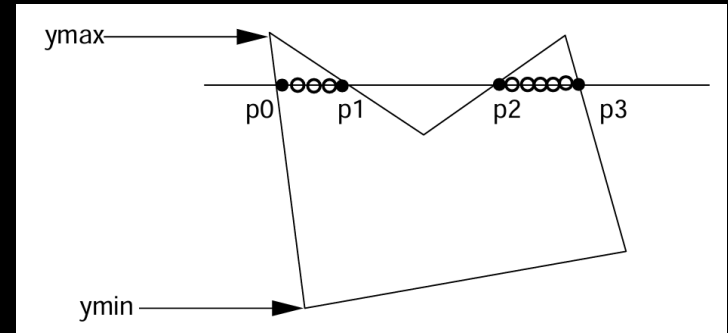
Advantages: It is relatively simple and easy to implement.

Disadvantages: It may not be efficient for complex polygons.

Algorithm:

For $y = y_{min}$ to y_{max}

- 1) intersect scanline y with each edge
- 2) sort intersections' by increasing x [p_0, p_1, p_2, p_3]
- 3) fill pairwise ($p_0 \rightarrow p_1, p_2 \rightarrow p_3, \dots$)



Boundary Fill Algorithm

Area Fill Algorithm: Boundary Fill Algorithm

- **Description:** This algorithm starts at a seed point inside a polygon and recursively fills neighboring pixels until a boundary color is encountered. It checks whether a pixel needs to be filled by comparing its color with the boundary color and the fill color.

Boundary Fill Algorithm is recursive in nature.

- ✓ It takes an interior point(x, y), a fill color, and a boundary color as the input.
- ✓ The algorithm starts by checking the color of (x, y). If it's color is not equal to the fill color and the boundary color, then it is painted with the fill color and the function is called for all the neighbours of (x, y).
- ✓ If a point is found to be of fill color or of boundary color, the function does not call its neighbours and returns. This process continues until all points up to the boundary color for the region have been tested.

Area Fill Algorithm: Boundary Fill Algorithm

Advantages

- It is easy to implement.
- Handles complex, irregular shapes
- Low memory for small spaces
- Lower overhead than flood fill
- Highly customizable

Disadvantages

- Risk of Stack overflow
- Slow for larger areas
- Strict boundary constraints

Area Fill Algorithm: Boundary Fill Algorithm

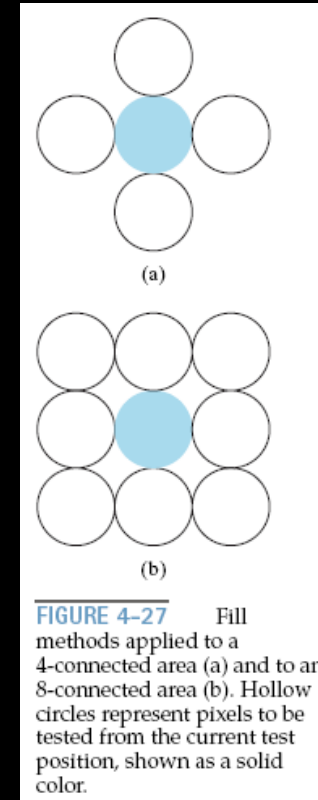
- The boundary fill algorithm can be implemented by 4-connected pixels or 8-connected pixels.

1. Four neighboring points or 4-connected pixels

- These are the pixel positions that are right, left, above, and below the current pixel.
- Areas filled by this method are called **4-connected**.

2. Eight neighboring points or 8-connected pixels

- This method is used to fill more complex figures.
- Here the set of neighboring points to be set includes the four diagonal pixels, in addition to the four points in the first method.
- Fill methods using this approach are called **8-connected**.



Area Fill Algorithm: Boundary Fill Algorithm

Four neighboring points or 4-connected pixels

- These are the pixel positions that are right, left, above, and below the current pixel.
- Areas filled by this method are called **4-connected**.

```
void boundaryFill4(int x, int y, int fill_color, int boundary_color)
{
    if(getpixel(x, y) != boundary_color &&
       getpixel(x, y) != fill_color)
    {
        putpixel(x, y, fill_color);
        boundaryFill4(x + 1, y, fill_color, boundary_color);
        boundaryFill4(x, y + 1, fill_color, boundary_color);
        boundaryFill4(x - 1, y, fill_color, boundary_color);
        boundaryFill4(x, y - 1, fill_color, boundary_color);
    }
}
```

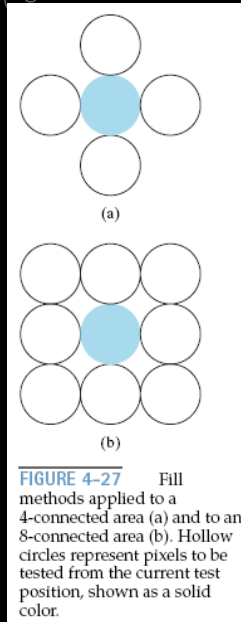
Area Fill Algorithm: Boundary Fill Algorithm

Eight neighboring points or 8-connected pixels

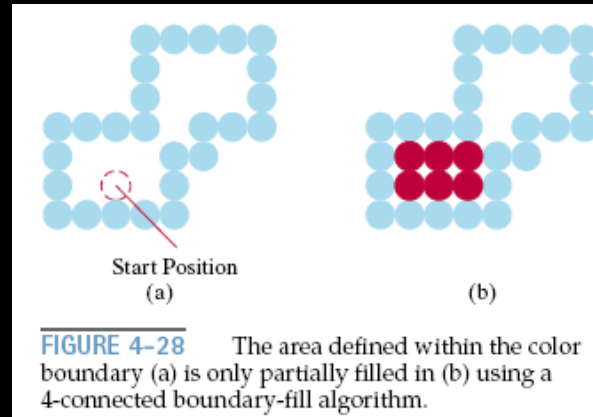
- This method is used to fill more complex figures.
- Here the set of neighboring points to be set includes the four diagonal pixels, in addition to the four points in the first method.
- Fill methods using this approach are called **8-connected**.

```
void boundaryFill8(int x, int y, int fill_color, int boundary_color)
{
    if(getpixel(x, y) != boundary_color &&
       getpixel(x, y) != fill_color)
    {
        putpixel(x, y, fill_color);
        boundaryFill8(x + 1, y, fill_color, boundary_color);
        boundaryFill8(x, y + 1, fill_color, boundary_color);
        boundaryFill8(x - 1, y, fill_color, boundary_color);
        boundaryFill8(x, y - 1, fill_color, boundary_color);
        boundaryFill8(x - 1, y - 1, fill_color, boundary_color);
        boundaryFill8(x - 1, y + 1, fill_color, boundary_color);
        boundaryFill8(x + 1, y - 1, fill_color, boundary_color);
        boundaryFill8(x + 1, y + 1, fill_color, boundary_color);
    }
}
```


Area Fill Algorithm: Boundary Fill Algorithm



- Consider the Figure in the below.
- An 8-connected boundary-fill algorithm would correctly fill the interior of the area defined in the Figure.
- But a 4-connected boundary-fill algorithm would only fill part of that region.



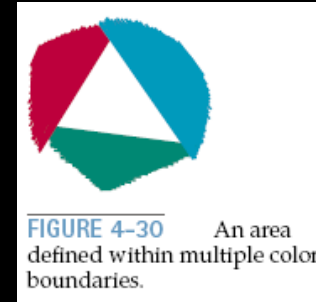
Flood Fill Algorithm

Area Fill Algorithm: Flood Fill Algorithm

- **Description:** Similar to the boundary fill algorithm, flood fill starts at a seed point and fills connected pixels until a boundary is encountered. It uses a *stack or a queue* to keep track of pixels to be filled.
- The flood fill algorithm has many characters similar to boundary fill. But this method is more suitable for filling multiple colors boundary. When boundary is of many colors and interior is to be filled with one color we use this algorithm.

Area Fill Algorithm: Flood Fill Algorithm

- Some times we want to fill in (or recolor) an area that is not defined within a single color boundary.
- Consider the following Figure.



- It takes an interior point(x, y), a fill color, and a target color (old color) as the input.
- The algorithm starts by checking the color of (x, y). If its color is equal to the target color and not equal to the fill color, then it is painted with the fill color and the function is called for all the neighbours of (x, y).
- If a point is found to be not of the target color or already of the fill color, the function does not call its neighbours and returns. This process continues until all connected points of the target color for the region have been tested.

Area Fill Algorithm: Flood Fill Algorithm

Advantages

- Multi-Colored Boundaries
- Perfect for Paint Bucket Tools
- Intuitive and Easy to Code

Disadvantages

- Fails completely on gradients
- High Memory Consumption

Area Fill Algorithm: Flood Fill Algorithm

```
void floodfill (x, y, fill_color, old_color: integer) {  
    if (getpixel (x, y) == old_color)  
    {  
        setpixel (x, y, fill_color);  
        fill (x+1, y, fill_color, old_color);  
        fill (x-1, y, fill_color, old_color);  
        fill (x, y+1, fill_color, old_color);  
        fill (x, y-1, fill_color, old_color);  
    }  
}
```

Boundary Fill Vs Flood Fill Algorithm

Boundary Fill Algorithm	Flood Fill Algorithm
Area filling is started inside a point with in a boundary region and fill the region with in the specified color until it reaches the the boundary.	Area filling is started from a point and it replaces the old color with the new color
It is used in interactive packages where we can specify the region boundary	It is used when we cannot specify the region boundary
It is less time consuming	It consumes more time
It searches for boundary.	It searches for old color

Inside Outside Test

Inside Outside Test

- The **Inside/Outside Test** is a technique used in **Computer Graphics** to determine whether a given point lies *inside a polygon or outside a polygon*.
- It is mainly used in polygon filling, clipping, collision detection, point selection, computer-aided design (CAD) and GIS (Geographical Information Systems).

There are mainly **two methods**:

- Even-Odd Rule (Odd-Even Rule)
- Winding Number Method

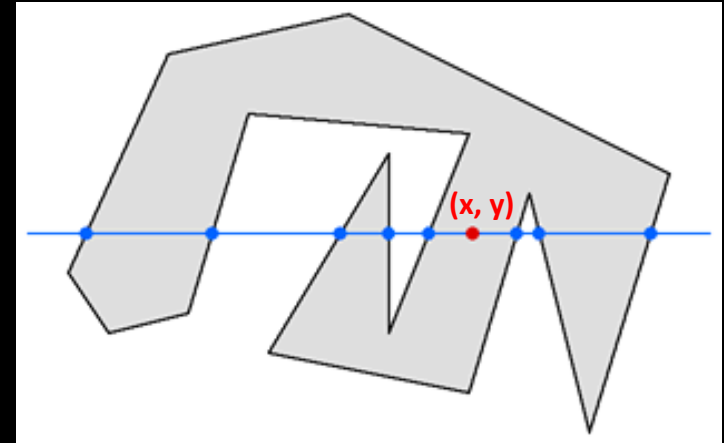
Inside Outside Test : Even odd rule

The Even-Odd Rule projects a conceptual line (ray) from the test point to infinity in any direction and counts how many polygon edges it intersects.

- **Odd Number of Intersections:** The point is **inside** the polygon.
- **Even Number of Intersections:** The point is **outside** the polygon.

Inside Outside Test : Even odd rule

- From the above figure, we can see that from the point (x, y) , the number of intersections point on the left side is 5 and on the right side is 3.
- From both ends, the number of intersection points is *odd*, so the point is considered within the object.



Questions

1. Explain Bresenham's line drawing algorithms. Compare it with DDA line algorithm. Draw a line from (2, 3) to (10, 8) by using Bresenham's algorithm. (LQ)
2. Derive the expression for Bresenham's Line Drawing algorithm. Trace the points in the line path with starting point (6, 12) and end point (10, 5) using Bresenham's line drawing algorithm. (LQ)
3. Explain the working details of Mid-point circle algorithm? Trace the points along the line path having two end points (6,9) and (2,3) using Bresenham's line drawing algorithm. (LQ)
4. What is a Digital Differential Analyzer (DDA)? How can you draw the line using this algorithm? Trace the points on the line path between A (1,3) and B (6,7) using Bresenham's line drawing algorithm. (LQ)
5. List the major differences between DDA and Bradenham's Line drawing algorithm. Illustrate the DDA algorithm to the line width end points (2, 2) and (9, 2). (LQ)
6. Plot the first octant of a circle centered at (-2, -2), having a radius of 5 units using the mid-point circle algorithm.
7. Write down algorithm steps of mid-point ellipse drawing algorithm.
8. Explain the importance of filling algorithms in graphics applications. Differentiate between boundary and flood fill algorithm with algorithm.
9. Plot the 1st octant of a circle centered at origin, having the radius 10 units.
10. Write short notes on: scanline polygonal filling technique, inside – outside test

THANK YOU
Any Queries ?